

APACHE DORIS 3.0.8

# 学习笔记

## 第六章：索引、缓存与高并发查询

主线：索引不是让计算凭空变快，而是用写入、空间和维护成本换取查询时少读数据；主键点查则进一步删除通用 OLAP 链路中不必要的工作。

配套编号：notes-06

基准版本：Apache Doris 3.0.8

整理日期：2026-08

# 目录

|                             |   |
|-----------------------------|---|
| 目录                          | 2 |
| 本章学习目标                      | 4 |
| 成本模型                        | 4 |
| 6.1 Doris 索引体系整体认知          | 4 |
| 索引不是一个单一结构                  | 4 |
| 本地索引与 MPP 的配合               | 5 |
| 两个判断维度                      | 5 |
| 6.2 排序键与前缀索引                | 5 |
| 从排序到导航                      | 5 |
| 36 字节与 VARCHAR 边界           | 6 |
| 设计思想                        | 6 |
| 6.3 ZoneMap 与 BloomFilter   | 6 |
| ZoneMap：用边界证明不可能命中          | 6 |
| BloomFilter：判断一定不存在         | 6 |
| 为什么会有假阳性                    | 6 |
| 和 Runtime Filter 的区别        | 6 |
| 6.4 Bitmap 与倒排索引            | 7 |
| Bitmap：值到行集合                | 7 |
| 倒排索引：Term 到 Posting List    | 7 |
| Doris 3.0.8 中的分词选择          | 7 |
| 历史数据构建                      | 7 |
| 6.5 NGram BloomFilter 与文本查询 | 7 |
| 把字符串切成固定长度片段                | 7 |
| 可下推才有意义                     | 8 |
| 6.6 索引下推、组合与 Profile 验证     | 8 |
| 多个索引是逐层缩小，而不是互相替代           | 8 |
| EXPLAIN 回答路径，Profile 回答效果   | 8 |
| 倒排索引关键指标                    | 8 |
| 常见不命中原因                     | 9 |
| 6.7 高并发点查为什么需要专用路径          | 9 |
| 固定成本压过数据读取                  | 9 |

|                                                                            |           |
|----------------------------------------------------------------------------|-----------|
| 为什么需要 Unique Key MOW . . . . .                                             | 9         |
| 严格适用边界 . . . . .                                                           | 9         |
| 复合主键示例 . . . . .                                                           | 9         |
| <b>6.8 Row Store、Row Cache、PreparedStatement 与 Short-Circuit . . . . .</b> | <b>10</b> |
| 行列混存解决宽表随机 IO . . . . .                                                    | 10        |
| Doris 3.0 的部分列行存 . . . . .                                                 | 10        |
| Page Cache 与 Row Cache . . . . .                                           | 10        |
| PreparedStatement 减少 FE CPU . . . . .                                      | 10        |
| 两条验证证据 . . . . .                                                           | 10        |
| <b>6.9 索引成本与表结构设计 . . . . .</b>                                            | <b>11</b> |
| 能力与代价地图 . . . . .                                                          | 11        |
| 从查询模板反推 . . . . .                                                          | 11        |
| 选择性决定上限 . . . . .                                                          | 11        |
| 冲突负载的物理拆分 . . . . .                                                        | 11        |
| <b>6.10 综合案例与索引失效诊断 . . . . .</b>                                          | <b>12</b> |
| 电商混合负载 . . . . .                                                           | 12        |
| 点查表关键属性 . . . . .                                                          | 12        |
| 分析表关键布局 . . . . .                                                          | 12        |
| 点查诊断顺序 . . . . .                                                           | 12        |
| 普通索引诊断矩阵 . . . . .                                                         | 12        |
| 最终方法论 . . . . .                                                            | 13        |
| <b>第六章复习题 . . . . .</b>                                                    | <b>13</b> |
| <b>参考资料 . . . . .</b>                                                      | <b>13</b> |

## 本章学习目标

前五章建立了架构、存储、执行、数据模型和物化计算的基础。第六章研究另一条加速主线：如何在读取数据之前尽可能排除无关数据，以及为什么高并发主键点查需要完全不同的存储和执行路径。

| 问题域  | 应能回答                     | 核心对象               |
|------|--------------------------|--------------------|
| 索引定位 | Doris 索引为何多是本地数据跳过结构？    | Segment、Page、RowID |
| 有序裁剪 | 排序键怎样同时服务压缩和前缀定位？        | Short Key、ZoneMap  |
| 集合过滤 | Bloom、Bitmap、倒排分别适合什么？   | 假阳性、位图、Posting     |
| 文本查询 | 全文检索和子串搜索为何不是一回事？        | Parser、NGram       |
| 点查路径 | 为什么列存 MPP 不适合高 QPS 单行查询？ | MOW、Short-Circuit  |
| 验证治理 | 怎样证明索引有效并权衡长期成本？         | EXPLAIN、Profile    |

分析查询的数据缩减漏斗



## 成本模型

~ + + + +  
~ - -

## 6.1 Doris 索引体系整体认知

Doris 的核心问题不是先找到一行，而是在海量列存数据中尽早证明哪些分区、Segment、Page 或 RowID 不必读取。

### 索引不是一个单一结构

| 层级   | 典型结构                | 排除粒度           |
|------|---------------------|----------------|
| 表级   | 分区裁剪                | 整个 Partition   |
| 有序数据 | 前缀索引                | 有序行范围          |
| 列页   | ZoneMap、BloomFilter | Segment 或 Page |
| 行集合  | Bitmap、倒排索引         | RowID 候选集      |
| 点查   | 主键索引、短路路径           | 一个主键对应的最新行     |

## 本地索引与 MPP 的配合

Doris 数据按分区和分桶拆成 Tablet，Tablet 副本分布在 BE。多数索引与各自 Segment 数据共同维护，FE 先裁剪宏观范围，BE 再使用本地索引缩小读取。它不是一个中央全局 B+Tree 承担所有查询，而是让每个并行扫描任务少做工作。

### 索引嵌入分布式执行



## 两个判断维度

- 谓词语义：等值、范围、低基数集合、全文分词、任意子串，所需结构不同。
  - 数据粒度：跳过一整个分区的收益通常大于只过滤少量 RowID，应优先从大粒度裁剪开始。
- 因此设计顺序通常是分区、分桶、排序键，再考虑辅助索引；不能用二级索引补救所有基础表结构问题。

## 6.2 排序键与前缀索引

排序键先决定物理有序性，前缀索引再从有序数据中抽取稀疏导航项；加速来自缩小连续读取范围。

### 从排序到导航

(dt, tenant\_id, user\_id)

2026-08-18, 1, 100  
2026-08-18, 1, 101  
2026-08-18, 2, 200  
2026-08-19, 1, 100

->

查询 dt 和 tenant\_id 的连续前缀时，可以快速定位窄区间；只按 user\_id 查询时，前导列未确定，同一个 user\_id 可能散落在许多区间。

| 条件                       | 前缀利用 | 原因                |
|--------------------------|------|-------------------|
| dt = ?                   | 好    | 命中第一个排序列          |
| dt = ? AND tenant_id = ? | 很好   | 连续前缀更长            |
| tenant_id = ?            | 弱    | 缺少首列              |
| dt BETWEEN ...           | 可用   | 定位连续范围，后续列能力受范围影响 |
| user_id = ?              | 通常弱  | 位于后部且前导列未知        |

## 36 字节与 VARCHAR 边界

Doris 的前缀索引只取排序 Key 的前 36 字节，并在遇到 VARCHAR 后形成明显边界。因此高频固定长度过滤列通常应靠前，长字符串不应轻易放在最前面。具体表仍需用真实数据和 EXPLAIN 验证。

### 设计思想

- 排序键不是把所有过滤字段按热度简单排列，而是在主查询族之间选择最有价值的共同前缀。
- 有序性还会增强 ZoneMap：同类值聚集后，很多页的 Min/Max 区间变窄。
- 为次要查询族需要的后部列，可考虑倒排等辅助索引，而不是破坏主排序顺序。

## 6.3 ZoneMap 与 BloomFilter

### ZoneMap：用边界证明不可能命中

列式 Page 或数据块会维护 Min、Max、Null 等统计。查询 amount > 1000 时，若某页 Max=80，就能跳过整个页；若 Min=20、Max=5000，只能继续读取。ZoneMap 没有精确指出目标行，只低成本排除。

```
Page A: min=1,    max=80    -> amount > 1000
Page B: min=500,  max=1500  ->
Page C: min=2000, max=9000  ->
```

如果值随机分布，每页都可能出现很小和很大的值，Min/Max 区间会非常宽；如果数据按该列聚集或近似有序，页区间更窄，裁剪效果更好。

### BloomFilter：判断一定不存在

BloomFilter 用多个哈希函数把集合成员映射到位数组。探测值时，只要有一个对应位为 0，就能确定它不存在；所有位都是 1，只能说可能存在。

BloomFilter 不返回精确行



### 为什么会有假阳性

不同值可能把相同位设置为 1。BloomFilter 不会把真实存在的值判为不存在，但可能把不存在的值判为可能存在。假阳性只损失性能，不应改变查询正确性。

$$p \approx (1 - e^{(-kn/m)})^k$$

nmk

### 和 Runtime Filter 的区别

| 对象                  | 何时产生                  | 服务对象                 |
|---------------------|-----------------------|----------------------|
| 表 BloomFilter 索引    | 数据写入/Compaction 时长期维护 | 某列的等值过滤与数据跳过         |
| Join Runtime Filter | 查询执行期间由 Build 端动态生成   | 把 Join 条件下推到 Probe 端 |

## 6.4 Bitmap 与倒排索引

### Bitmap：值到行集合

```
status = PAID    -> 0011010010...
region = EAST    -> 0111000010...
PAID AND EAST    -> 0011000010...
```

位图把值对应的 RowID 集合编码成比特位，AND、OR、NOT 可以用 CPU 位运算快速组合。它适合状态、地区、设备类型等中低基数字段；如果几乎每行都是独立值，会产生大量稀疏集合和元数据，未必经济。

### 倒排索引：Term 到 Posting List

正向数据是 RowID 到文本；倒排结构反过来记录词项到包含它的 RowID 列表。全文查询不再逐行解析所有文本，而是读取词项对应 Posting List，再做交集、并集或短语位置判断。

```
row 1: payment timeout
row 2: login failed
row 3: payment failed
```

```
payment -> [1, 3]
failed  -> [2, 3]
payment AND failed -> [3]
```

### Doris 3.0.8 中的分词选择

| parser  | 适合内容      | 注意                 |
|---------|-----------|--------------------|
| english | 英文单词和标点   | 大小写、词边界需实测         |
| chinese | 中文文本      | 可选择粗粒度或细粒度模式       |
| unicode | 多语言通用文本   | 语义效果应以 TOKENIZE 验证 |
| none    | 整串值、非全文语义 | 适合等值而非中文分词搜索       |

短语查询还依赖位置数据，应根据需求设置 support\_phrase。分词器决定索引里保存什么词；查询词与建索引分词不一致时，索引存在也可能得不到预期语义。

### 历史数据构建

CREATE INDEX 后，新写入数据可以携带倒排索引；已有历史 Segment 需要执行 BUILD INDEX 并检查构建状态。把创建元数据误认为历史数据已全部建好，是常见故障来源。

## 6.5 NGram BloomFilter 与文本查询

全文倒排回答词项语义；NGram BloomFilter 主要帮助排除不可能包含某段字符的数据块，最终仍需 LIKE 验证。

### 把字符串切成固定长度片段

```
gram_size = 3
payment -> pay, aym, yme, men, ent
```

```
LIKE '%yme%' yme Bloom
yme ->
```

它仍具有 BloomFilter 的假阳性：可能读取一个最终不匹配的数据块，但不会因为过滤而漏掉真实匹配。gram\_size 越小，短片段覆盖更广但冲突更多；越大，区分度提高，但短搜索串可能无法有效利用。bf\_size 增大通常降低冲突，同时增加空间。

| 需求           | 更适合         | 原因              |
|--------------|-------------|-----------------|
| 包含某个英文或中文词   | 倒排索引        | 需要分词语义          |
| 多个词全部出现      | 倒排索引        | Posting List 交集 |
| 精确短语         | 支持短语的倒排索引   | 需要词位置信息         |
| LIKE '%abc%' | NGram BF    | 任意位置字符串         |
| 完整字符串等值      | 前缀/Bloom/倒排 | 依据布局 and 选择性决定  |

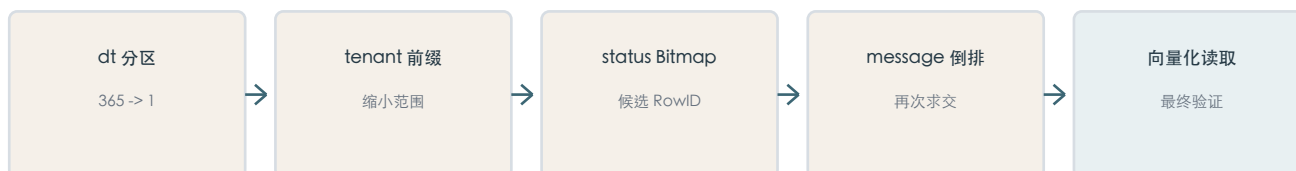
## 可下推才有意义

索引必须在扫描前被存储层使用才会减少读取。对列套复杂函数、隐式类型转换或不受支持的表达式，可能让谓词无法下推。NGram LIKE 场景应检查 enable\_function\_pushdown，并以 EXPLAIN/Profile 为准。

## 6.6 索引下推、组合与 Profile 验证

### 多个索引是逐层缩小，而不是互相替代

组合过滤示例



理想组合先使用便宜且排除粒度大的条件，再用更精确的 RowID 结构，最后读取必要列完成残余谓词。执行器会基于可用条件和成本选择，用户不应假设每个索引都必然被使用。

### EXPLAIN 回答路径，Profile 回答效果

| 工具            | 主要问题               | 示例证据                                |
|---------------|--------------------|-------------------------------------|
| EXPLAIN       | 计划是否裁剪、谓词是否下推、是否短路 | partition、predicates、SHORT-CIRCUIT  |
| Query Profile | 实际读了多少、过滤多少、耗时在哪里  | RowsRead、RowsReturned、索引过滤行数、I/O 时间 |

### 倒排索引关键指标

- RowsInvertedIndexFiltered：倒排索引排除的行数。
- InvertedIndexFilterTime：倒排过滤总耗时。
- InvertedIndexSearcherOpenTime：打开搜索器的时间。
- InvertedIndexSearcherSearchTime：内部索引检索时间。

过滤行数大不代表整条 SQL 必然快；Join、聚合、排序、Exchange、宽列读取和巨大结果集都可能成为下一瓶颈。



### 常见不命中原因

- 历史数据索引仍未构建完成。
- 谓词形式不受支持，或列被 CAST、函数和隐式转换包裹。
- 分词器、parser\_mode 或短语设置与查询语义不一致。
- 条件选择性太低，优化器判断扫描更便宜。
- 索引确实生效，但剩余数据和后续算子仍然很大。

## 6.7 高并发点查为什么需要专用路径

点查返回的数据极少，通用 OLAP 链路的解析、规划、Fragment 和调度固定成本反而成为主体；优化关键是删除这些工作。

### 固定成本压过数据读取

= SQL + + + Fragment  
          + + + +

= + Tablet + RPC + +

Short-Circuit 快路径



### 为什么需要 Unique Key MOW

同一主键可能持续写入 created、paid、shipped 等版本。MOW 在写入阶段维护最新版本，使读取阶段不必临时合并多个版本。它用写入成本换取可预测的最新行定位，是短路读取的存储前提。

### 严格适用边界

| 条件     | 要求                                               |
|--------|--------------------------------------------------|
| 表模型    | Unique Key，enable_unique_key_merge_on_write=true |
| 表属性    | light_schema_change=true，建表时启用 Row Store         |
| 谓词     | 全部 Key 列均为 AND 连接的等值条件                           |
| SQL 形态 | 单表，无 Join、嵌套子查询、聚合和范围扫描                          |
| 验证     | EXPLAIN 出现 SHORT-CIRCUIT                         |

### 复合主键示例

```
UNIQUE KEY(tenant_id, order_id)

WHERE tenant_id = ? AND order_id = ?
WHERE order_id = ?
WHERE order_id BETWEEN ? AND ? ->
```

这是一条受约束的 KV 式读取能力，不等于 Doris 具备完整 OLTP 数据库的事务、锁和任意二级索引点查模型。

## 6.8 Row Store、Row Cache、PreparedStatement 与 Short-Circuit

### 行列混存解决宽表随机 IO

列存分析只读少数列很高效，但 SELECT \* 点查一行可能访问许多列页再拼装。启用 Row Store 后，Doris 在列存之外增加一份紧凑行编码，点查可以用一次行页读取获得多个字段。

同一数据的两种物理读取方式



### Doris 3.0 的部分列行存

```

PROPERTIES (
  "enable_unique_key_merge_on_write" = "true",
  "light_schema_change" = "true",
  "row_store_columns" = "order_id,status,amount,update_time"
)
  
```

row\_store\_columns 从 3.0 起允许只编码在线接口稳定返回的热列，降低全列行存的空间和写放大。行存必须在建表阶段规划；字段选择应来自真实 API 契约。

### Page Cache 与 Row Cache

| 缓存         | 缓存单位    | 主要负载 | 风险            |
|------------|---------|------|---------------|
| Page Cache | 列式 Page | 分析扫描 | 大扫描会带来淘汰压力    |
| Row Cache  | 主键对应的行  | 热点查  | 占用独立内存，需要控制容量 |

```

BE
disable_storage_row_cache=false
row_cache_mem_limit=20%
  
```

### PreparedStatement 减少 FE CPU

同一 SQL 模板只改变主键参数时，服务端 PreparedStatement 可以在会话内复用已解析语句、表达式和描述信息。收益依赖连接和 PreparedStatement 对象的复用；每次重连并重新 prepare 会破坏复用。

```

jdbc:mysql:loadbalance://fe1:9030,fe2:9030/demo
?useServerPrepStmts=true&cachePrepStmts=true
&prepStmtCacheSize=500&prepStmtCacheSqlLimit=1024
  
```

```
SELECT order_id, status, amount FROM order_serving WHERE order_id = ?
```

### 两条验证证据

1. EXPLAIN 计划中出现 SHORT-CIRCUIT，证明走专用执行路径。
2. fe.audit.log 中出现 Stmt=EXECUTE(...)，证明服务端 PreparedStatement 在工作。

## 6.9 索引成本与表结构设计

正确问题不是某列能否加索引，而是某个稳定查询模板是否值得长期支付写入、存储、Compaction 和运维成本。

### 能力与代价地图

| 能力           | 主要收益          | 主要代价                |
|--------------|---------------|---------------------|
| 分区/排序        | 排除大范围，形成有序聚集  | 物理布局需提前设计           |
| Bloom/Bitmap | 低成本排除或集合过滤    | 空间、假阳性或基数边界         |
| 倒排/NGram     | 文本、等值、范围或子串加速 | 索引文件、写入与 Compaction |
| Row Store    | 宽表整行点查少 IO    | 数据冗余和写放大            |
| Row Cache    | 热点点查低延迟       | BE 内存与淘汰治理          |

### 从查询模板反推

1. 收集高频且高优先级 SQL，按过滤、返回列、排序和 Join 归类。
2. 确认谓词语义和选择性：等值、范围、全文、子串、主键点查。
3. 优先设计分区、分桶和排序键，再选择最小必要辅助索引。
4. 用 EXPLAIN 验证路径，用 Profile 比较过滤量、读取量和耗时。
5. 将导入、Compaction、空间和查询收益放在同一观察周期评估。

### 选择性决定上限

= /  
1 10  
1 6000

### 冲突负载的物理拆分

高频点查、全文搜索、大范围聚合和高吞吐写入会要求不同冗余结构。可以让同一实时事实流写入主键服务表、明细分析表和聚合表/物化视图，使每张表围绕一个主要访问模式优化。

#### 访问模式建模

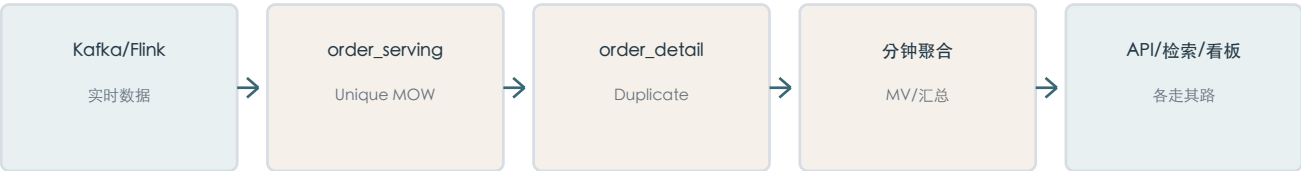


## 6.10 综合案例与索引失效诊断

### 电商混合负载

系统既要按订单号读取最新状态，又要按日期和商户分析明细、搜索备注、匹配回调 URL，并维护分钟级实时看板。将所有能力堆在一张表会同时放大行存、倒排、NGram、写入和 Compaction 成本。

推荐物理模型



#### 点查表关键属性

```
UNIQUE KEY(order_id)
DISTRIBUTED BY HASH(order_id) BUCKETS 32
PROPERTIES (
  "enable_unique_key_merge_on_write" = "true",
  "light_schema_change" = "true",
  "row_store_columns" = "order_id,user_id,status,amount,update_time"
)
```

#### 分析表关键布局

```
DUPLICATE KEY(dt, merchant_id, order_id)
PARTITION BY RANGE(dt)
DISTRIBUTED BY HASH(merchant_id) BUCKETS 32
remark      -> INVERTED(parser=chinese, support_phrase=true)
callback_url -> NGRAM_BF(gram_size=3, bf_size=1024)
```

### 点查诊断顺序

1. EXPLAIN 是否出现 SHORT-CIRCUIT。
2. SHOW CREATE TABLE 是否为 Unique MOW，并已启用 light\_schema\_change 和 Row Store。
3. WHERE 是否完整覆盖全部 Key 列，且只有等值条件。
4. 是否存在 Join、子查询、聚合、函数包裹或类型转换。
5. JDBC 是否使用服务端 PreparedStatement；Audit Log 是否出现 EXECUTE。
6. 路径正确后，再区分首次存储读取与 Row Cache 命中问题。

### 普通索引诊断矩阵

| 症状              | 优先检查                                  |
|-----------------|---------------------------------------|
| 索引过滤为 0         | 谓词支持、历史构建、分词、类型转换和函数下推                |
| 过滤很多仍很慢         | 返回规模、宽列、Join、聚合、排序、Exchange           |
| 新数据有效旧数据无效      | BUILD INDEX 和构建状态                     |
| 大扫描后点查变慢        | Row Cache、内存容量和淘汰压力                   |
| FE CPU 高而 BE 空闲 | PreparedStatement、短路路径、Observer 和负载均衡 |

## 最终方法论

先识别查询形态，再选择数据布局；先证明路径生效，再分析实际收益；先消除大粒度数据，再优化剩余计算。

## 第六章复习题

建议先不看答案，分别用一句话结论、底层数据结构、因果链和验证手段回答。

1. 为什么 Doris 的索引体系更强调数据跳过，而不是中央全局 B+Tree？
2. 排序键与前缀索引分别承担什么职责？
3. 为什么只按排序键后部列过滤时，前缀索引效果通常较弱？
4. ZoneMap 为什么在数据聚集时更有效？
5. BloomFilter 为什么可以有假阳性，却不能有假阴性？
6. 表级 BloomFilter 与 Join Runtime Filter 的生命周期有何不同？
7. Bitmap 索引为什么更适合中低基数字段？
8. 倒排索引怎样把全文扫描变成 Posting List 查询？
9. 全文倒排索引和 NGram BloomFilter 的文本语义有何不同？
10. 为什么索引存在不等于谓词一定能够下推？
11. EXPLAIN 与 Query Profile 在索引诊断中分别回答什么问题？
12. 为什么列式存储适合分析，却不天然适合宽表整行点查？
13. 普通 OLAP 链路中的哪些固定成本会限制点查 QPS？
14. 为什么 Short-Circuit 要求完整主键等值条件和 Unique Key MOW？
15. Row Store 在底层如何减少宽表点查的随机 IO？
16. row\_store\_columns 解决了什么成本问题？
17. Page Cache、Row Cache 与 PreparedStatement 分别优化哪一层？
18. 怎样分别证明 Short-Circuit 和服务端 PreparedStatement 已经生效？
19. 为什么索引过滤很多行后，整条 SQL 仍可能很慢？
20. 面对同时包含点查、全文检索和聚合的实时业务，怎样进行物理模型拆分？

## 参考资料

本文以 Apache Doris 3.0.8 为基准。3.x 文档用于核对 3.0 起能力时，仅采用与 3.0.8 兼容的部分；后续版本新增能力不纳入本章。

[1] Doris 3.x - Index Best Practice

[2] Doris 3.x - Inverted Index

[3] Doris 3.x - CREATE INDEX

[4] Doris 3.x - Row Store

[5] Doris - High-Concurrency Point Query

[6] Doris 3.x - Profile Action

[7] Release 3.0.8